实验报告

第 01 讲　计算机核心结构 · 数制转换与存储容量估算

实验名称	**数制转换与存储容量估算**		
对应章节	第 1 章 计算机原理	**实验学时**	2 学时
班级		**姓名**	
学号		**日期**	
实验工具	纸笔 / Python / 计算器		
成绩			

一、实验目的

（1）数制转换：能对 8 位二进制数与 0–255 的十进制整数完成二、十、十六进制互转，逐位写出位权与位权展开式，并用 Python 的 int() / bin() / hex() 复核，结果与手算完全一致。

（2）容量推导：能由“分辨率 × 位深”“采样率 × 位深 × 声道数 × 时长”“码率 × 时长”三组公式，独立推导照片、音频、视频的未压缩体积，并在 SI（10^3）与 IEC（2^{10}）两套单位下正确换算（相对误差 ＜0.1%）。

（3）验证与解释：能用实测文件体积对比理论值、量化压缩比，说清“压缩后的体积不能当作未压缩估算结果”，并能对 256 GB 手机给出可存照片数量与取舍建议。

二、实验环境与工具

硬件环境	CPU：13th Gen Intel Core i7-13700H（14 核 20 线程）；内存 32 GB；显卡 NVIDIA GeForce RTX 4060 Laptop GPU（另有 Intel Iris Xe 核显）；硬盘 NVMe SSD。
软件与版本	Windows 11（10.0.26200）；Windows PowerShell 5.1；Python 3.12.14；Pillow 12.3.0；LibreOffice Kit（导出 PDF）。
数据与素材	实验指南 PDF；测试图 assets/（73.1 KB、3.66 MB）；实测数据 results/压缩对比.csv；源码 estimate.py 与单元测试。

三、实验步骤与过程记录

序号	操作与命令	参数或输入	结果与现象
1	手算二进制 → 十进制（按权展开）	10110101_2	位权依次为 128、64、32、16、8、4、2、1；非零项 128+32+16+4+1 = **181**，与 0xB5 一致
2	手算十进制 → 十六进制（除 16 取余）	255	255÷16 = 15 余 15(F)；15÷16 = 0 余 15(F)；余数倒序排列得 **FF**
3	手算十六进制 → 二进制（一位拆四位）	$1A_{16}$	1 → 0001，A → 1010；拼接 0001 1010，去掉前导 0 得 **11010**
4	Python 复核三组结果	int('10110101',2)、hex(255)、int('1A',16)、bin(0x1A)	依次输出 181、0xff、26、0b11010，与手算逐位一致
5	估算未压缩照片	1920×1080、24 bit	6,220,800 B = **6.2208 MB** = **5.9326 MiB**（约 6.2 MB / 5.9 MiB）
6	估算 PCM 音频	44.1 kHz、16 bit、双声道、4 min	42,336,000 B = **42.336 MB** = **40.3748 MiB**（约 42 MB）
7	按码率估算视频	8 Mbps、2 h	57,600,000,000 bit ÷ 8 = **7,200,000,000 B** = **7.2 GB**（6.7055 GiB）
8	压缩实测：同一内容换编码	BMP / PNG / JPEG q95 / JPEG q75	界面图 PNG 73.1 KB 反小于 JPEG q95 237.7 KB；照片 JPEG q75 154.2 KB，为未压缩的 1/40
9	脚本自检与回归	python estimate.py check；python -m unittest	7 项断言全部 PASS；15 个单元测试全部通过

四、实验结果

本节的图 1–图 5 与表 1–表 3 对应实验指南的 7 个步骤：图 1、图 2 是脚本运行输出，图 3 与表 3 是压缩实测，图 4 用手机截图核对分辨率与实际体积，图 5 是脚本自检；表中数值保留 4 位小数单位同时给出 SI 与 IEC 两种口径。

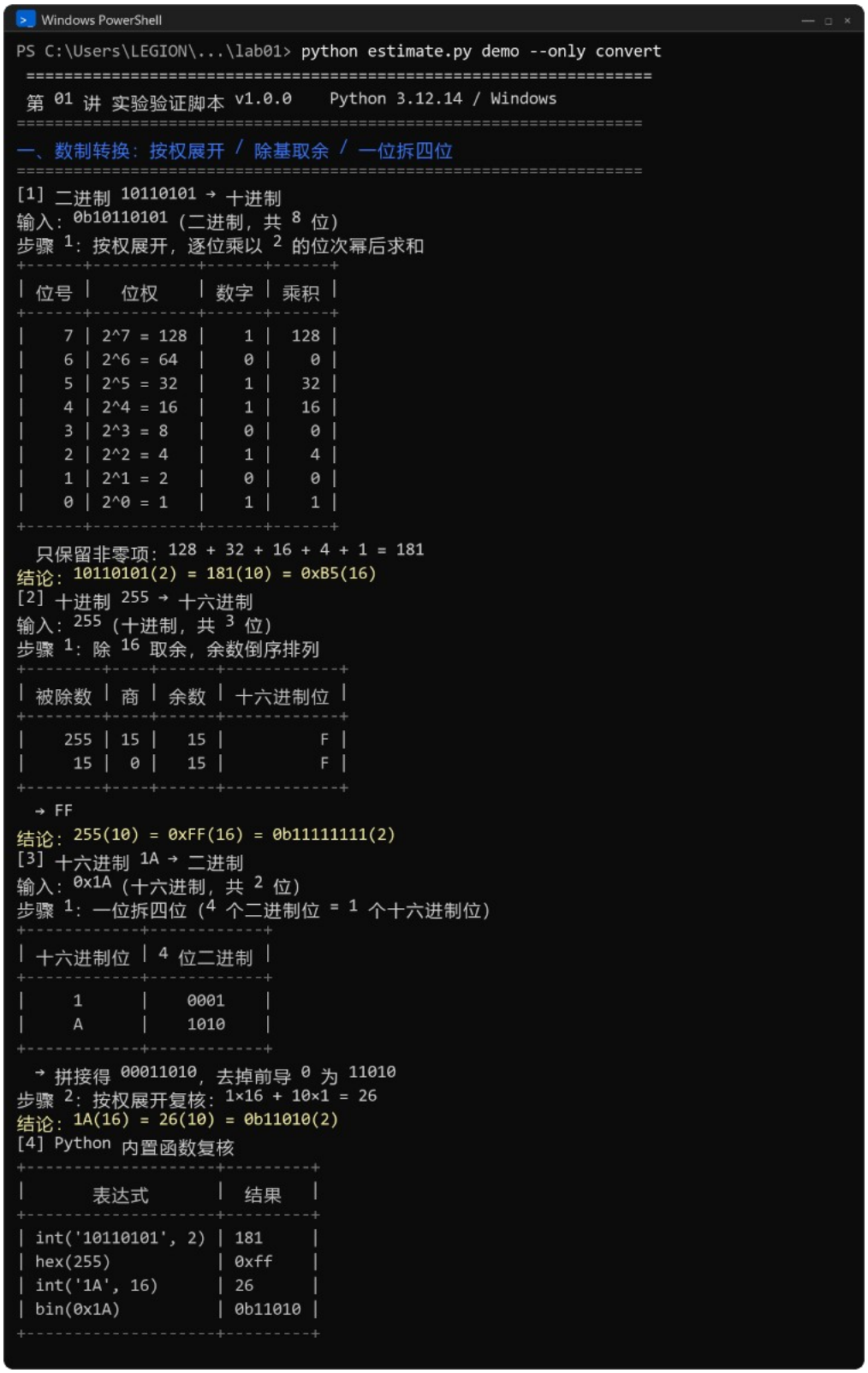


图 1 数制转换运行结果（python estimate.py demo --only convert）

图 1 中三组转换的位权、余数与结论均由脚本逐步打印，[4] 一节用 int() / hex() / bin() 复核；手算与程序结果逐位一致，说明位权展开与除基取余两种方法可以互相验算。

表 1 数制转换计算过程表

题目	方法	过程	结果
10110101_2 → 十进制	按权展开	$1\times2^7+0\times2^6+1\times2^5+1\times2^4+0\times2^3+1\times2^2+0\times2^1+1\times2^0$ = 128+32+16+4+1	**181**
255 → 十六进制	除 16 取余	255÷16 = 15 余 15(F)；15÷16 = 0 余 15(F)，余数倒序	**FF_{16}**
$1A_{16}$ → 二进制	一位拆四位	1 → 0001，A → 1010；0001 1010 去前导 0	**11010_2**
三项复核	Python 内置函数	int('10110101',2)=181；hex(255)='0xff'；int('1A',16)=26；bin(0x1A)='0b11010'	全部一致

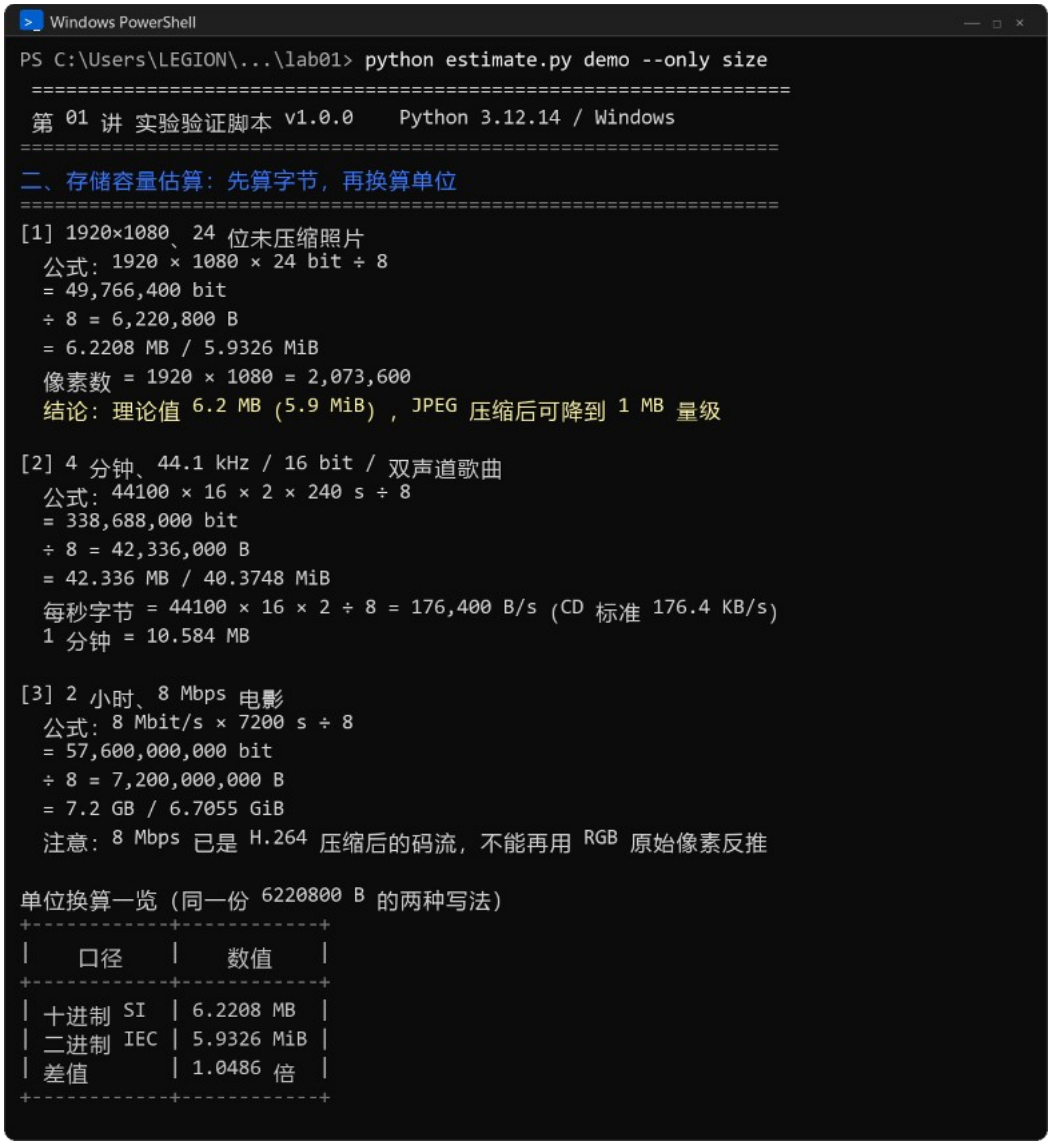


图 2 容量估算运行结果（python estimate.py demo --only size）

图 2 中每个对象都按“公式 → bit → B → SI / IEC”四步打印，最后一张小表把同一份 6,220,800 B 写成 6.2208 MB 与 5.9326 MiB，直观显示两种进位制的 1.048576 倍差。

表 2 容量估算计算过程表（含公式与单位换算）

对象	公式	代入计算	字节数 / B	SI 口径	IEC 口径
照片 1920×1080×24 bit	宽×高×位深÷8	1920×1080×24÷8	6,220,800	6.2208 MB	5.9326 MiB
音频 4 min / 44.1 kHz / 16 bit / 双声道	采样率×位深×声道×时长÷8	44,100×16×2×240÷8	42,336,000	42.336 MB	40.3748 MiB
视频 2 h / 8 Mbps	码率×时长÷8	$8\times10^6\times7200\div8$	7,200,000,000	7.2 GB	6.7055 GiB
对照：1 min CD 音频	采样率×位深×声道×60÷8	44,100×16×2×60÷8	10,584,000	10.584 MB	10.0937 MiB



图 3 同一内容、不同编码的文件体积对比（数据见 results/压缩对比.csv）

表 3 压缩实测数据（括号内为相对未压缩体积的压缩比）

内容	理论未压缩	BMP（实测）	PNG	JPEG q95	JPEG q75
界面截图 1080×2400	7.78 MB	7.78 MB	73.1 KB（106.3×）	237.7 KB（32.7×）	101.5 KB（76.6×）
照片 1920×1080	6.22 MB	6.22 MB	3.66 MB（1.7×）	1.35 MB（4.6×）	154.2 KB（40.4×）

表 3 中 BMP 实测比理论值多 54 B，是 BMP 文件头；PNG 对界面图几乎无损却只有 73.1 KB，对照片却要 3.66 MB；JPEG 在照片上以 4.6 倍压缩仍保持 q95 质量。可见编码效率高度依赖内容，“压缩后的体积”不能反推“未压缩体积”。



图 4 手机课表截图（1206×1068，用于核对分辨率与实际文件大小）

图 4 为本人手机截图，裁切后像素为 1206×1068。按 24 位计，未压缩应为 1206×1068×3 B = 3,864,024 B ≈ 3.86 MB，而实际 JPEG 文件只有 84.6 KB，约为未压缩的 1/45.7——与表 3 中“界面截图 PNG 73.1 KB”是同一类现象：大面积纯色与锐利文字让 PNG 与 JPEG 都能大幅压缩。（该文件经聊天软件转发时已二次压缩，45.7 倍仅作数量级参考。）可见截图分辨率只能给出未压缩上限，实际体积由编码与内容共同决定。



图 5 脚本自检结果（python estimate.py check），7 项断言全部 PASS

作品或数据文件路径：lab01/estimate.py、lab01/tests/、lab01/results/压缩对比.csv、lab01/assets/（报告与 PDF 见 lab01/docs/）

五、问题与解决

问题	原因分析	解决办法
同一张照片按 1024 进位算得 5.9 MB，与指南“约 6.2 MB”对不上；4 分钟音频同样差 1.96 MB	MB 有两套口径：SI 的 10^6 B 与 IEC 的 2^{20} B。指南给的是 6.2 MB（10^6），课件第 46 页写 5.9 MB（实为 MiB），两处混用。	以字节 B 为唯一中间量，输出时并列两种口径：“6.2208 MB / 5.9326 MiB”；check 同时断言 6.2 MB 与 5.9 MiB。验证：两者恒差 1.048576 倍，7 项自检全部 PASS（图 5）。
视频 8 Mbps × 7200 s 先算得 57.6 Gbit，若直接写成 57.6 GB，比参考答案大 8 倍	码率单位是 bit/s，文件大小用 Byte，漏了“÷8”；bit 与 Byte 差 8 倍是容量题最常见的错误。	公式固定为“码率×时长÷8”，输出按 bit → B → GB 三级打印中间量；再用数量级自查（2 小时电影约 7 GB，不可能是 57 GB）。
十六进制 1A 转二进制时写成 101，与 int('1A',16)=26 对不上	A 对应 1010 四位，只写 101 丢了最高位；十六进制转二进制必须一位拆四位、不足四位左补 0。	按“一位拆四位”重写为 0001 1010，去前导 0 得 11010；用 bin(0x1A) 复核通过，并把该例写进单元测试。

六、结论与拓展

（1）数制转换：位权展开与除基取余互为逆运算，十六进制与二进制按“一位拆四位”互转。$10110101_2 = 181 = 0xB5$，$255 = FF_{16}$，$1A_{16} = 26 = 11010_2$；三组结果与 int() / hex() / bin() 完全一致，目标（1）达成。

（2）容量推导：三条公式覆盖三类媒体——像素×位深÷8、采样率×位深×声道×时长÷8、码率×时长÷8；分别得 6.2208 MB、42.336 MB、7.2 GB，与指南参考答案（6.2 MB / 42 MB / 7.2 GB）一致，目标（2）达成。

（3）容量直觉：一张 1080p 未压缩照片约 6 MB，一首 4 分钟 CD 音质歌曲约 42 MB（≈7 张照片），一部 2 小时 8 Mbps 电影约 7.2 GB（≈170 首歌）。记住三个数量级——照片 MB、歌曲十 MB、电影 GB——就能在交卷前完成自查。

（4）压缩的影响：未压缩体积只由“像素×位深”决定，是存储上限；实际文件大小由编码决定。同一张界面图，PNG 73.1 KB 反而小于 JPEG q95 237.7 KB；同一张照片，JPEG q75 154.2 KB 只有未压缩的 1/40。因此估算必须先分清“理论未压缩”与“实际落盘”，不能用压缩后的体积反推位深或分辨率。

（5）拓展：按 256 GB 标称、系统占用 12% 计，可用 225.28 GB（209.81 GiB）。单张 4 MB 可存 56,320 张；改用 4800 万像素 12 MB 只剩 18,773 张；4K / 100 Mbps 视频 1 小时 45 GB，仅够 5.0 小时。取舍：日常记录用 4 MB 档并开启云端备份，重要素材再上高像素与高码率；拍摄前按“码率×时长÷8”算账，给系统与备份留 10%–15% 余量。

七、提交物自查

- ☑ 计算过程表（含公式与单位换算）
- ☑ 验证脚本 estimate.py
- ☑ 一页结论：容量直觉与压缩的影响

其他附件：lab01/ 源码包、实验报告 PDF；Git 仓库 / 网盘链接：_____

八、评分（教师填写）

项目	分值	得分	说明
数制转换	30		过程正确、位权书写规范
容量推导	40		公式正确、单位换算无误
验证与解释	20		脚本可运行，能解释压缩差异
报告规范	10		表格清晰、结论明确
合计	**100**		百分制，由评阅教师填写

评阅教师：_____ **日期：**_____

附录 A 关键代码（estimate.py 节选）

完整源码见 lab01/estimate.py（约 330 行，含 CLI 与 15 个单元测试）。以下节选核心算法与自检逻辑。

```
def parse_int(text: str, base: int) -> int:
    """把任意进制的字符串转成十进制整数（支持 0b / 0o / 0x 前缀与下划线）。"""
    s = text.strip().replace("_", "")
    prefix = {"0b": 2, "0o": 8, "0x": 16}
    for head, b in prefix.items():
        if s.lower().startswith(head):
            s, base = s[2:], b
            break
    if not 2 <= base <= 16:
        raise ValueError(f"进制必须在 2-16 之间，收到 {base}")
    value = 0
    for ch in s.upper():
        digit = DIGITS.index(ch) if ch in DIGITS else -1
        if digit < 0 or digit >= base:
            raise ValueError(f"字符 {ch!r} 不是 {base} 进制的合法数字：{text!r}")
        value = value * base + digit
    return value

def to_base(value: int, base: int) -> str:
    """十进制整数转 base 进制字符串（除基取余、余数倒序）。"""
    if value < 0:
        return "-" + to_base(-value, base)
    if value == 0:
        return "0"
    out = []
    while value:
        value, rem = divmod(value, base)
        out.append(DIGITS[rem])
    return "".join(reversed(out))

def image_bytes(width: int, height: int, depth: int = 24) -> int:
    """未压缩位图：宽 × 高 × 位深 ÷ 8。depth=24 即 RGB 各 8 位。"""
    return width * height * depth // 8

def audio_bytes(seconds: float, rate: int = 44100, depth: int = 16, channels: int = 2) -> int:
    """未压缩 PCM：采样率 × 位深 × 声道数 × 时长 ÷ 8。"""
    return int(rate * depth * channels * seconds) // 8

def video_bytes(seconds: float, mbps: float) -> int:
    """按码率估算：码率(Mbit/s) × 时长(s) ÷ 8。"""
    return int(mbps * 10**6 * seconds) // 8

def human_si(n: float) -> str:
    for name, factor in reversed(SI_UNITS):
        if abs(n) >= factor:
            return f"{n / factor:.4f}".rstrip("0").rstrip(".") + f" {name}"
    return f"{n:.0f} B"

def human_iec(n: float) -> str:
    for name, factor in reversed(IEC_UNITS):
        if abs(n) >= factor:
            return f"{n / factor:.4f}".rstrip("0").rstrip(".") + f" {name}"
    return f"{n:.0f} B"

def check() -> tuple[bool, str]:
    """把关键结果与实验指南的参考答案逐条比对。"""
    img = image_bytes(1920, 1080, 24)
    aud = audio_bytes(240, 44100, 16, 2)
    vid = video_bytes(7200, 8)
    cases = [
        ("10110101(2) → 十进制", 181, parse_int("10110101", 2)),
        ("255 → 十六进制", "FF", to_base(255, 16)),
        ("1A(16) → 十进制", 26, parse_int("1A", 16)),
        ("1920×1080×24bit", "6.2 MB", f"{img / 10**6:.1f} MB"),
        ("同一照片（IEC）", "5.9 MiB", f"{img / 2**20:.1f} MiB"),
        ("4 min / 44.1k / 16bit / 双声道", "42.3 MB", f"{aud / 10**6:.1f} MB"),
```

```
    ("2 h / 8 Mbps", "7.2 GB", f"{vid / 10**9:.1f} GB"),
]
rows = []
ok = True
for name, want, got in cases:
    passed = str(want) == str(got)
    ok &= passed
    rows.append([name, want, got, "PASS" if passed else "FAIL"])
text = table(["检查项", "参考答案", "脚本结果", "判定"], rows, "<^^^")
return ok, text
```

附录 B 复现与提交说明

· **环境**：Python 3.12+，依赖仅 numpy 与 Pillow（生成测试图）；无需安装第三方库即可运行 estimate.py。

· **验证**：在 lab01/ 目录执行 python estimate.py check（7 项断言）与 python -m unittest discover -s tests（15 个用例）。

· **复现**：python tools/make_test_images.py 重新生成测试图与 results/压缩对比.csv；python tools/render_terminal.py 把任意命令输出渲染为报告插图。

· **提交**：lab01/ 已按 Git 仓库规范组织（README.md、LICENSE、.gitignore）。本地推送命令见 提交说明.md，仓库地址与网盘链接填入第七节“其他附件”。